

CS193

Homework - Week 3

Exploring Real Weather Data from the Command Line

Introduction

In lecture you learned how to go beyond typing one command at a time and start **combining commands** in bash — finding files, and chaining tools together with **pipes and filters**. This week you'll put those skills to work on a real dataset.

The data comes from **WHIN** (the [Wabash Heartland Innovation Network](#)), which runs a dense network of weather stations across a 10-county region of north-central Indiana. You'll work with a fixed snapshot of readings from **24 stations** and answer questions about it entirely from the command line.

You only need **bash and a terminal** — no other languages, no installs. A helpful companion reference is *The Linux Command Line* (linuxcommand.org/tlcl.php), especially the chapters on navigation and I/O redirection.

Getting Started

Download and unzip the `cs193_week3` folder. It already contains everything you need:

- `whin_stations.csv` — the snapshot, with **one row per station**. Columns are: `station_id`, `station_name`, `county`, `latitude`, `longitude`, `temp_f`, `humidity_pct`, `wind_mph`, `rain_in`, `soil_moist_pct`, `observed_at`.
- `station_logs/` — one log file per station, each with that station's hourly readings.

How to Work Through It

1. Open a terminal (in VS Code: **Terminal** → **New Terminal**, or any terminal app).
2. `cd` into the project folder so the file names below work:

```
$ cd cs193_week3
```

3. Run each command below, read the output, and record your answer. No server, compiler, or installation is required — it's plain bash.

Part A — Finding Your Way Around (7 min)

Every station writes its own log file. Let's locate them.

- A1.** List the log files, then count them. How many stations logged data today?

```
$ ls station_logs
$ ls station_logs | wc -l
```

- A2.** Now count *every* CSV file in the folder, including the snapshot. `find` searches inside sub-folders for you.

```
$ find . -name "*.csv" | wc -l
```

- A3.** WHIN's Tippecanoe County stations all start with TIP. List just those log files.

```
$ find station_logs -name "TIP*"
```

- A4.** Print the full path of every **White County** log file (their IDs start with WHI).

```
$ find station_logs -name "WHI*"
```

Part B — Reading the Data with Pipes & Filters (10 min)

Now work with the snapshot file `whin_stations.csv`.

- B1.** Peek at the first 5 lines. What does the very first line contain?

```
$ head -5 whin_stations.csv
```

- B2.** How many stations are in the snapshot? The header row is not a station, so skip it with `tail -n +2` (“start at line 2”).

```
$ tail -n +2 whin_stations.csv | wc -l
```

- B3.** Print just the station names (column 2). The file is comma-separated, so tell `cut` the delimiter is a comma.

```
$ cut -d, -f2 whin_stations.csv
```

- B4.** How many stations are in **Tippecanoe** County? `grep -c` counts matching lines.

```
$ grep -c Tippecanoe whin_stations.csv
```

- B5.** Count how many stations are in *each* county. Chain four tools: take the county column, sort it, then collapse duplicates with a count.

```
$ tail -n +2 whin_stations.csv | cut -d, -f3 | sort | uniq -c
```

- B6.** Which station is the **coolest** in this snapshot? Temperature is column 6. Sort the rows numerically by that column and take the first one.

```
$ tail -n +2 whin_stations.csv | sort -t, -k6 -n | head -1
```

Challenge (optional): Write a **single command line** that prints only the station names in **White** County. Hint: `grep` to pick the rows, then pipe into `cut` to keep the name column.

How to Submit

1. Paste each command you ran and its output into a plain-text file named `week3_lastname.txt`.
2. At the bottom, add one sentence answering: *“Which county has the most stations, and how did you find out?”*
3. Go to Ed Discussion and open the **Week 3** submission thread.
4. Create a new post, attach your `week3_lastname.txt`, and submit.

Grading

- **A1** — count the log files with `ls | wc -l` — 10 pts
- **A2** — count all CSVs with `find ... | wc -l` — 10 pts
- **A3** — list TIP* files with `find` — 10 pts
- **A4** — list WHI* files with `find` — 10 pts
- **B1** — `head`; identifies the first line as the header row — 8 pts
- **B2** — skips the header with `tail -n +2` and counts with `wc -l` — 8 pts
- **B3** — extracts the name column with `cut -d, -f2` — 8 pts
- **B4** — counts Tippecanoe rows with `grep -c` — 8 pts
- **B5** — `cut | sort | uniq -c` chain for per-county counts — 10 pts
- **B6** — `sort -t, -k6 -n | head -1` for the coolest station — 8 pts
- **Submission & reflection** — correctly named file with commands, outputs, and the reflection sentence — 10 pts
- **Challenge (bonus)** — `grep White | cut -d, -f2` in one line — +5 pts

Total: 100 points (+5 bonus).